

TinyMPC with Differential Flatness: α -Parameterized Templates for Efficient Quadrotor Control

Jiangnan Zhang
Dartmouth College

Abstract—TinyMPC enables real-time model-predictive control on microcontrollers by caching the infinite-horizon Riccati solution and requiring only matrix–vector products at each ADMM iteration. However, its linearization is fixed at a single operating point (typically hover), which degrades tracking performance for aggressive trajectories. We exploit the *differential flatness* of quadrotor dynamics to reparameterize the linearization by a flat parameter $\alpha = (a_x, a_y, a_z, \psi)$. This yields closed-form equilibria and Jacobians—no automatic differentiation required—making the computation FPGA-portable. We present two extensions of TinyMPC: (i) *Flat Template MPC*, which precomputes DARE caches at a grid of α values for online nearest-neighbor lookup, and (ii) *Analytic Template*, a zero-iteration state-feedback controller using first-order gain interpolation (~ 260 arithmetic operations per control output). Preliminary Python simulations on a Crazyfly-scale quadrotor show that, on figure-eight trajectory tracking, the Flat Template achieves tracking accuracy within 6% of online relinearization (0.035 m average position error) at $29\times$ lower wall-clock time, while the Analytic Template maintains 0.036 m error with zero ADMM iterations.

I. INTRODUCTION

Model-predictive control (MPC) is a powerful framework for controlling highly dynamic robots subject to state and input constraints [1]. TinyMPC [1] demonstrated that, by caching the infinite-horizon discrete algebraic Riccati equation (DARE) solution offline and performing only matrix–vector products online, the ADMM-based MPC solver can run on microcontrollers with as little as 192 kB of RAM at rates exceeding 500 Hz.

A key assumption in TinyMPC is that the dynamics are linearized at a *fixed operating point*—typically the hover equilibrium for a quadrotor. This is reasonable for near-hover regulation but becomes a limitation for aggressive trajectory tracking, where the true linearization varies significantly along the reference. The standard remedy—online relinearization at each timestep—requires Jacobian evaluation (via automatic differentiation or finite differences) and a fresh DARE solve, both of which are computationally expensive and ill-suited for resource-constrained platforms.

We observe that quadrotor dynamics are *differentially flat* with flat output $\mathbf{y} = (p_x, p_y, p_z, \psi)$ [2], [3]. This classical property has been widely used for trajectory generation [2], [4], but its implications for *MPC solver structure* have received less attention. We show that flatness enables a closed-form parameterization of both the equilibrium and the Jacobian by

a single four-dimensional *flat parameter* $\alpha = (a_x, a_y, a_z, \psi) \in \mathbb{R}^4$, representing the commanded translational acceleration and yaw angle. Because the resulting expressions involve only elementary trigonometric and algebraic operations, they require no automatic differentiation and are directly implementable on FPGAs.

Crucially, because α is four-dimensional—compared to the $n = 12$ dimensional state—it serves as a natural *scheduling variable* for the DARE solution. This connects our approach to classical gain scheduling [11] and linear parameter-varying (LPV) control [12], but with the scheduling space compressed from $\mathbb{R}^n$ to $\mathbb{R}^4$ by the flat map. We make this connection precise in Section III-C and show that the Analytic Template is equivalent to a first-order gain schedule over α , where the gain sensitivity is obtained from perturbation analysis of the DARE [13], [14].

Building on this observation, we propose two extensions of TinyMPC:

- 1) **Flat Template MPC** (Section III-B): Precompute DARE caches at a grid of α values offline. Online, look up the nearest cache and run K ADMM iterations as in standard TinyMPC, but with α -specific matrices $(A_d, B_d, K_\infty, P_\infty, C_1, C_2)$.
- 2) **Analytic Template** (Section III-C): Precompute the gain K_0 at hover and its four partial derivatives $\partial K / \partial \alpha_i$ offline. Online, interpolate $K(\alpha) \approx K_0 + \sum_i \delta \alpha_i \partial_i K$ and apply a single feedback step—no ADMM iterations, no constraint projection. The total cost is ~ 260 multiply-accumulate operations.

Both variants shift the coordinate frame to the flat equilibrium $(x^*(\alpha), u^*(\alpha))$, so the ADMM solver always operates in a neighborhood of zero error—regardless of the trajectory aggressiveness. Fig. 1 illustrates the pipeline and the relationship among the four controller variants.

Contributions

- An explicit, autograd-free linearization of quadrotor dynamics parameterized by $\alpha \in \mathbb{R}^4$ (Section III-A).
- Flat Template MPC: an α -indexed cache family for TinyMPC’s ADMM solver (Section III-B).
- Analytic Template: a zero-iteration controller with first-order gain approximation (Section III-C).

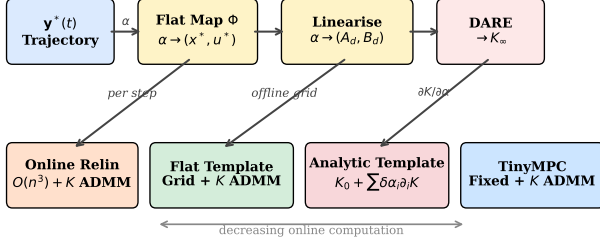


Fig. 1. Controller hierarchy. The flat map Φ and analytic linearization (yellow) are shared by all flat-parameterized variants. Moving right trades online computation for approximation: Online Relinearization re-solves DARE per step; Flat Template looks up a precomputed cache; the Analytic Template interpolates $K(\alpha)$ in ~ 260 ops.

- Preliminary numerical validation comparing four controller variants on hover and figure-eight tracking (Section IV).

II. BACKGROUND AND RELATED WORK

A. Related Work

Gain scheduling. The classical approach to controlling nonlinear systems around varying operating points is gain scheduling [11]: solve the linear control problem (e.g., LQR or H_∞) at a grid of operating points offline, then interpolate the resulting gains online. Rugh and Shamma [11] survey the extensive theory, including conditions under which interpolated controllers inherit stability from the frozen-parameter designs. Our Flat Template MPC is a gain-scheduled ADMM-MPC controller where the scheduling variable is the flat parameter α , and our Analytic Template is a first-order gain schedule with an explicit interpolation formula (11).

LPV control. Linear parameter-varying (LPV) systems generalize gain scheduling by treating the varying linearization as a parameter-dependent plant $\dot{x} = A(\theta)x + B(\theta)u$ where $\theta(t)$ is measurable in real time [12]. Our framework fits this structure with $\theta = \alpha$, but we exploit flatness to guarantee that α is a *four-dimensional sufficient statistic* for the equilibrium—rather than requiring the full n -dimensional state as scheduling variable.

Riccati sensitivity analysis. The dependence of the DARE solution on system parameters has been studied since Hewer [13], with explicit perturbation bounds developed by Konstantinov et al. [14]. Our gain sensitivities $\partial K / \partial \alpha_i$ in (10) are a numerical instance of this theory, approximated by central differences. The key practical insight is that for quadrotor dynamics, $K(\alpha)$ varies smoothly and slowly with α , so a first-order expansion suffices (4% mean relative error over the tested range).

Flatness-based tracking control. Differential flatness has been widely used for quadrotor trajectory generation [2], [4] and feedforward computation [10]. Mellinger and Kumar [2] use the flat map to compute reference states and inputs, then track with a fixed-gain controller. Our approach extends this by additionally using the flat map to parameterize the *feedback*

gain itself, adapting K to the current operating condition through the DARE sensitivity.

Explicit MPC. An alternative to online optimization is explicit MPC [15], which precomputes the optimal control law as a piecewise-affine function of the state via multi-parametric programming. The Flat Template MPC shares the “precompute offline, look up online” philosophy, but avoids the exponential complexity of explicit MPC by storing only DARE caches (not full polyhedral partitions) and running a small number of ADMM iterations online.

B. TinyMPC

TinyMPC [1] solves the constrained linear MPC problem, formulated as a quadratic program (QP) [7], [9],

$$\begin{aligned} \min_{x_{1:N}, u_{1:N-1}} \quad & \frac{1}{2} x_N^\top Q_f x_N + \sum_{k=1}^{N-1} \left(\frac{1}{2} x_k^\top Q x_k + \frac{1}{2} u_k^\top R u_k \right) \\ \text{s.t.} \quad & x_{k+1} = A x_k + B u_k, \\ & x_k \in \mathcal{X}, \quad u_k \in \mathcal{U}, \end{aligned} \quad (1)$$

via ADMM. The key insight is that, for a sufficiently long horizon, the Riccati recursion converges to the infinite-horizon solution (K_∞, P_∞) . TinyMPC caches

$$\begin{aligned} K_\infty &= (R_\rho + B^\top P_\infty B)^{-1} B^\top P_\infty A, \\ C_1 &= (R_\rho + B^\top P_\infty B)^{-1}, \\ C_2 &= (A - B K_\infty)^\top, \end{aligned} \quad (2)$$

where $R_\rho = R + \rho I$ and $Q_\rho = P_{\text{qr}} + \rho I$ are the ADMM-augmented cost matrices. Each ADMM iteration then requires only matrix–vector products:

- (i) *Backward pass*: compute feedforward corrections $d_k = C_1(B^\top p_{k+1} + r_k)$, $p_k = q_k + C_2 p_{k+1} - K_\infty^\top r_k$.
- (ii) *Forward pass*: $u_k = -K_\infty x_k - d_k$, $x_{k+1} = A x_k + B u_k$.
- (iii) *Slack update*: project onto constraints.
- (iv) *Dual update*: gradient ascent on multipliers.

The entire online computation is *matrix-inversion free*. However, the matrices A, B (and hence all cached quantities) are fixed at a single linearization point.

C. Differential Flatness of Quadrotor Dynamics

A quadrotor with state $(p, \dot{p}, R, \omega) \in \mathbb{R}^3 \times \mathbb{R}^3 \times \text{SO}(3) \times \mathbb{R}^3$ and collective thrust T along the body z -axis satisfies

$$m\ddot{p} = T R e_3 - m g e_3. \quad (3)$$

The system is differentially flat with flat output $y = (p_x, p_y, p_z, \psi)$ [2], [3]. This means that every state and input can be expressed as an algebraic function of y and a finite number of its time derivatives.

The flat map proceeds as follows. Define the *total acceleration vector*

$$t = \ddot{p} + g e_3 = \begin{pmatrix} a_x \\ a_y \\ a_z + g \end{pmatrix}, \quad (4)$$

where (a_x, a_y, a_z) is the commanded translational acceleration in the world frame. Then:

- 1) **Thrust:** $T = m\|\mathbf{t}\|$.
- 2) **Attitude:** The body z -axis is $z_B = \mathbf{t}/\|\mathbf{t}\|$. Given yaw ψ , the rotation R is fully determined. In Euler angles (ZYX convention):

$$\phi^* = \arcsin((a_x \sin \psi - a_y \cos \psi)/\|\mathbf{t}\|), \quad (5)$$

$$\theta^* = \arctan 2(a_x \cos \psi + a_y \sin \psi, a_z + g). \quad (6)$$

- 3) **Angular velocity and torques:** determined by $p^{(3)}$ and $p^{(4)}$ (jerk and snap) through the kinematic and Euler equations; at equilibrium, $\omega^* = 0$.

We define the *flat parameter* $\alpha = (a_x, a_y, a_z, \psi) \in \mathbb{R}^4$, which fully specifies the equilibrium state and input:

$$\Phi : \alpha \mapsto (x^*(\alpha), u^*(\alpha)). \quad (7)$$

III. METHOD

A. Analytic Linearization

Given α , we construct the continuous-time Jacobians $A_c(\alpha), B_c(\alpha)$ of the 12-dimensional Euler-angle state $x = (p, \phi, \theta, \psi, v, \omega)$ about the flat equilibrium (7).

The 12×12 matrix A_c has the block structure

$$A_c = \begin{pmatrix} 0_{3 \times 3} & 0_{3 \times 3} & I_3 & 0_{3 \times 3} \\ 0_{3 \times 3} & 0_{3 \times 3} & 0_{3 \times 3} & W(\phi^*, \theta^*) \\ 0_{3 \times 3} & G(\alpha) & 0_{3 \times 3} & 0_{3 \times 3} \\ 0_{3 \times 3} & 0_{3 \times 3} & 0_{3 \times 3} & 0_{3 \times 3} \end{pmatrix}, \quad (8)$$

where $G(\alpha) \in \mathbb{R}^{3 \times 3}$ is the gravity coupling matrix (derivative of the thrust vector with respect to attitude at equilibrium) and $W(\phi^*, \theta^*)$ is the Euler-angle kinematic map relating body angular velocity to Euler rates.

The 12×4 matrix B_c has nonzero blocks in the velocity rows (thrust direction $\times k_t/m$) and the angular velocity rows (torque allocation $J^{-1}M$), where M is the motor mixing matrix.

Discretization via forward Euler yields

$$A_d = I_{12} + A_c \Delta t, \quad B_d = B_c \Delta t. \quad (9)$$

The entire computation is closed-form trigonometry and linear algebra, requiring no automatic differentiation.

B. Flat Template MPC

Flat Template MPC extends TinyMPC by maintaining a family of DARE caches indexed by α .

Offline. Select a grid $\{\alpha^{(j)}\}_{j=1}^M$ (e.g., sampled along the reference trajectory). For each grid point:

- 1) Compute $(A_d^{(j)}, B_d^{(j)}) = (A_d(\alpha^{(j)}), B_d(\alpha^{(j)}))$.
- 2) Solve DLQR to obtain terminal cost $P_{\text{lqr}}^{(j)}$.
- 3) Solve augmented DARE (2) to obtain $K_\infty^{(j)}, P_\infty^{(j)}, C_1^{(j)}, C_2^{(j)}$.
- 4) Store the full cache $\{A_d, B_d, K_\infty, P_\infty, C_1, C_2\}^{(j)}$.

Online. At each control step with current flat parameter α :

- 1) Look up the nearest cache entry $j^* = \arg \min_j \|\alpha - \alpha^{(j)}\|$.
- 2) Compute the flat equilibrium $(x^*, u^*) = \Phi(\alpha)$.
- 3) Form the error state $\delta x = x - x^*$.

- 4) Run K ADMM iterations using cache j^* (identical to TinyMPC's backward/forward/slack/dual passes).
- 5) Apply $u = u^* + \delta u_0$, where δu_0 is the first-step control from the ADMM solution.

This preserves TinyMPC's matrix-inversion-free online structure while adapting the linearization to the current operating point. The offline cost is M DARE solves (each ~ 5000 Riccati iterations), and the online lookup is $O(M)$ nearest-neighbor search.

C. Analytic Template

The Analytic Template eliminates ADMM iterations entirely, producing a control output in ~ 260 arithmetic operations.

Offline.

- 1) Set $\alpha_0 = (0, 0, 0, 0)$ (hover).
- 2) Solve the augmented DARE at α_0 to obtain the base gain $K_0 \in \mathbb{R}^{4 \times 12}$.
- 3) For $i = 1, \dots, 4$, compute gain sensitivities via central differences:

$$\frac{\partial K}{\partial \alpha_i} \approx \frac{K(\alpha_0 + \epsilon e_i) - K(\alpha_0 - \epsilon e_i)}{2\epsilon}. \quad (10)$$

- 4) Store K_0 and $\{\partial_i K\}_{i=1}^4$: a total of $5 \times 4 \times 12 = 240$ floating-point numbers.

Online. Given state x and flat parameter α :

- 1) $\delta \alpha = \alpha - \alpha_0$.
- 2) Compute $(x^*, u^*) = \Phi(\alpha)$.
- 3) Interpolate the gain (192 MACs):

$$K(\alpha) \approx K_0 + \sum_{i=1}^4 \delta \alpha_i \frac{\partial K}{\partial \alpha_i}. \quad (11)$$

- 4) Compute the control (48 MACs):

$$u = u^*(\alpha) - K(\alpha)(x - x^*(\alpha)). \quad (12)$$

- 5) (Optional) Clip to actuator bounds.

Relationship to TinyMPC. The Analytic Template output is *exactly equal* to the first forward-pass step of TinyMPC's ADMM iteration with zero warm-start (i.e., before any constraint information is incorporated). Specifically, when all ADMM dual and slack variables are initialized to zero, the first-iteration forward pass produces $u_0 = -K_\infty \delta x$, which matches (12) since $K(\alpha) \approx K_\infty(\alpha)$ from the same augmented DARE. Subsequent ADMM iterations incorporate constraint information through the slack and dual updates. Thus, the Analytic Template trades constraint handling for extreme computational efficiency.

Connection to gain scheduling and Riccati sensitivity. The control law (12) with the gain interpolation (11) is precisely a *first-order gain schedule* over the flat parameter α [11]. The gain sensitivities $\partial K / \partial \alpha_i$ are instances of the classical Riccati perturbation theory [13], [14], evaluated numerically via (10). What makes this gain schedule unusually compact is differential flatness: whereas a generic gain schedule over the n -dimensional state would require $O(n)$ sensitivity matrices, flatness reduces the scheduling space to

TABLE I
HOVER STABILIZATION ($N_{\text{sim}} = 100$, $\rho = 85$, $N = 10$).

	Tiny MPC	Online Relin	Flat Template	Analytic Template
Avg. pos. error (m)	0.064	0.095	0.138	0.145
Max pos. error (m)	0.346	0.346	0.346	0.346
Avg. ADMM iters	80.0	16.7	7.8	0
Total ADMM iters	8002	1669	785	0
Wall time (s)	2.85	12.13	0.44	0.13

$\mathbb{R}^4$, yielding exactly four sensitivity matrices regardless of state dimension. This produces the identity:

$$\underbrace{u^* - K(\alpha) \delta x}_{\text{Analytic Template}} = \underbrace{u^* - K_\infty(\alpha) \delta x}_{\text{ADMM iter. 0}} + O(\epsilon^2), \quad (13)$$

where the $O(\epsilon^2)$ term is the gain approximation error from the first-order Taylor expansion. The four variants thus form a chain, with each step trading optimality for computational savings:

$$\begin{aligned} \text{Online Relin} &\xrightarrow{\text{cache}} \text{Flat Template} \xrightarrow{K \rightarrow 0} \\ &\text{Analytic Template} \xrightarrow{\text{1st ord.}} \text{Gain Schedule.} \end{aligned}$$

IV. PRELIMINARY RESULTS

We validate the proposed methods in simulation using a Crazyflie 2.1 quadrotor model (mass 35 g, arm length 46 mm) with a control frequency of 50 Hz. All simulations use a Python reference implementation with NumPy linear algebra.

A. Controller Variants

We compare four controllers:

- 1) **TinyMPC** (baseline): Fixed linearization at hover, ADMM with warm-starting and convergence tolerance 10^{-3} .
- 2) **Online Relinearization**: Autograd-based Jacobian at each timestep, fresh DARE solve, then ADMM.
- 3) **Flat Template MPC**: α -parameterized analytic linearization with precomputed DARE cache grid, then ADMM.
- 4) **Analytic Template**: Zero-iteration state feedback with first-order gain interpolation.

B. Hover Stabilization

The quadrotor starts at $(0.2, 0.2, -0.2)$ m from the hover point. Horizon $N = 10$, penalty $\rho = 85$, $N_{\text{sim}} = 100$ steps.

At hover, TinyMPC's fixed linearization is exact and achieves the lowest average error (Table I, Fig. 2). However, it requires $10\times$ more ADMM iterations than the Flat Template due to the high ρ value. The Flat Template converges faster because its cache already incorporates the correct equilibrium structure. The Analytic Template achieves the lowest wall-clock time ($22\times$ faster than TinyMPC) at a modest accuracy penalty.

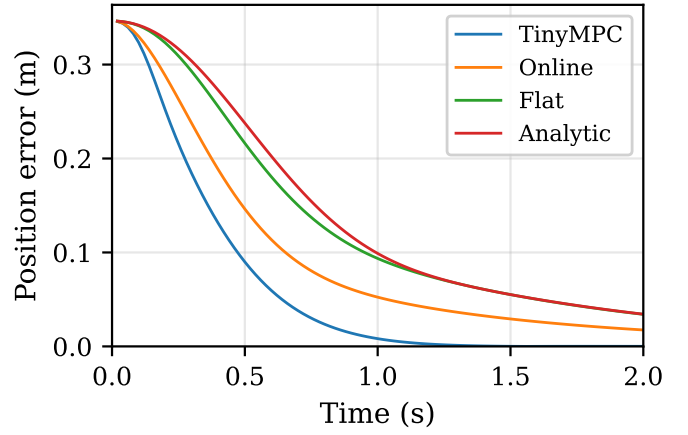


Fig. 2. Hover stabilization: position error over time for all four variants. Initial displacement $(0.2, 0.2, -0.2)$ m. All variants converge; the flat-parameterized controllers trade slightly slower convergence for significantly lower computation per step.

TABLE II
FIGURE-EIGHT TRAJECTORY TRACKING ($N_{\text{sim}} = 200$, $\rho = 5$, $N = 15$).

	Tiny MPC	Online Relin	Flat Template	Analytic Template
Avg. pos. error (m)	0.033	0.035	0.035	0.036
Max pos. error (m)	0.091	0.092	0.113	0.116
Avg. ADMM iters	8.0	5.1	3.6	0
Total ADMM iters	1598	1023	714	0
Wall time (s)	1.16	22.02	0.75	0.26

C. Figure-Eight Trajectory Tracking

The reference is a figure-eight in the xz -plane with amplitude 0.5 m and period 3.7 s. Horizon $N = 15$, penalty $\rho = 5$, $N_{\text{sim}} = 200$ steps. The Flat Template uses 21 cache entries sampled uniformly over one period.

On the figure-eight trajectory (Table II, Figs. 3–4), all four variants achieve similar average tracking errors (0.033–0.036 m), demonstrating that the flat-parameterized linearization is sufficiently accurate for moderate-aggressiveness trajectories. The key differences are computational:

- The Flat Template requires 55% fewer total ADMM iterations than TinyMPC and runs $1.5\times$ faster in wall-clock time.
- Online relinearization is $29\times$ slower than the Flat Template due to per-step autograd Jacobian evaluation and DARE solves, despite using fewer ADMM iterations.
- The Analytic Template achieves $4.5\times$ speedup over TinyMPC with only 0.003 m additional average error, using *zero* ADMM iterations.

D. Numerical Validation

We verify the implementation with four tests:

- 1) **Linearization consistency**: At hover, the analytic A_d, B_d match the autograd-based Jacobians to within 10^{-3} in position-velocity coupling and 10^{-15} in torque allocation.

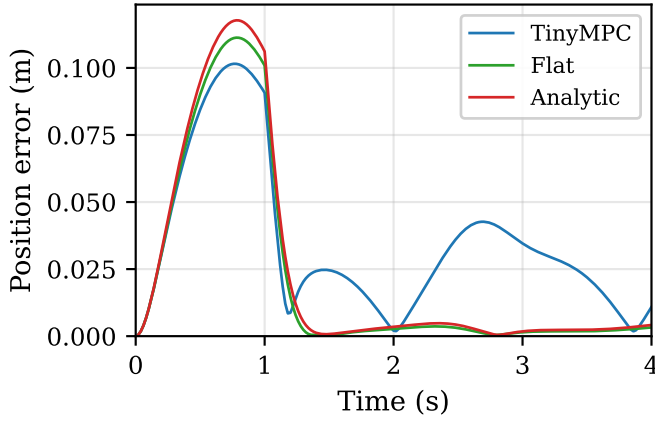


Fig. 3. Figure-eight tracking: position error over time. All variants achieve comparable accuracy; the periodic structure reflects the trajectory curvature.

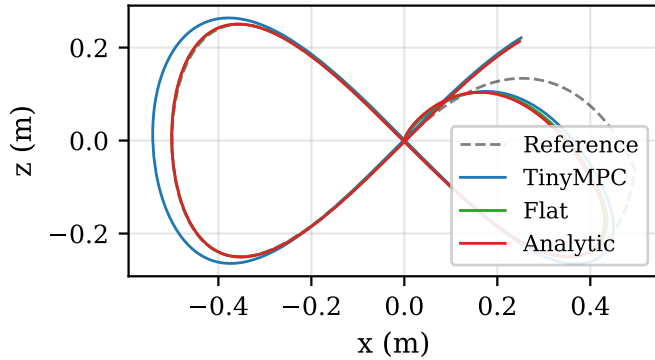


Fig. 4. Figure-eight tracking: x - z plane view. The Flat Template and Analytic Template closely follow the reference trajectory (dashed), comparable to TinyMPC.

Attitude-related entries differ by $\sim 2\times$ due to the Euler-angle vs. quaternion-error parameterization.

- 2) **Gain approximation:** Over 100 random α samples with $\|a\| \leq 3 \text{ m/s}^2$, the first-order gain approximation (11) achieves mean relative Frobenius error of 4%.
- 3) **Control equivalence:** At hover with inactive constraints, the Analytic Template output equals the 1-iteration Flat Template output to machine precision ($< 10^{-8}$).
- 4) **Flat equilibrium:** For 20 random α values, $f(x^*(\alpha), u^*(\alpha))$ produces the expected continuous-time derivatives to $< 10^{-14}$.

V. DISCUSSION

When does the fixed linearization suffice? For near-hover operation, TinyMPC’s fixed linearization is optimal by construction. The flat-parameterized variants become advantageous when the trajectory induces significant attitude variation—the equilibrium shift ensures the ADMM solver operates near zero error at every timestep, improving convergence.

Computational hierarchy. The four variants form a natural hierarchy (see also the equivalence chain (13)):

Variant	Online ops	Constraints	Scheduling
Online Relin	$O(n^3) + K \cdot O(n^2)$	Yes	$\mathbb{R}^n$
TinyMPC	$K \cdot O(n^2)$	Yes	fixed
Flat Template	$K \cdot O(n^2)$	Yes	$\mathbb{R}^4$ grid
Analytic	$O(n \cdot m)$	Clip only	$\mathbb{R}^4$ 1st order

The Flat Template has the same per-iteration cost as TinyMPC but converges in fewer iterations due to better-conditioned linearization. The Analytic Template trades constraint handling for extreme efficiency, making it suitable for the innermost loop of a hierarchical controller or for FPGA implementation.

What is new vs. what is classical. The individual mathematical ingredients—differential flatness [2], [5], gain scheduling [11], and Riccati sensitivity [13], [14]—are well established. Our contribution is their specific combination in the context of embedded ADMM-based MPC: (i) using flatness to compress the scheduling dimension from n to 4, (ii) caching the *augmented* DARE (with ADMM penalty ρ) rather than the standard LQR DARE, and (iii) showing the equivalence between the zero-iteration ADMM forward pass and the flat gain schedule. This combination yields the 240-float, 260-operation datapath of the Analytic Template—a level of compactness not achievable by generic gain scheduling or LPV approaches without the flatness reduction.

FPGA portability. The analytic linearization (Section III-A) and the Analytic Template (Section III-C) are composed entirely of elementary operations: trigonometric functions, multiplications, and additions. There are no loops with data-dependent iteration counts and no matrix inversions. The 240-float storage and ~ 260 -operation datapath are directly realizable as a fixed-function pipeline.

Limitations. The first-order gain approximation (11) degrades for large $\|\alpha - \alpha_0\|$. For highly aggressive maneuvers, the Flat Template MPC with a sufficiently dense cache grid is preferred. The current Python results do not reflect embedded timing; C/C++ implementation with Crazyflie hardware experiments is ongoing work.

VI. CONCLUSION

We presented two extensions of TinyMPC that exploit differential flatness to adapt the linearization point along a trajectory. The Flat Template MPC precomputes a family of DARE caches indexed by the flat parameter α , preserving TinyMPC’s matrix-inversion-free online structure while achieving faster ADMM convergence. The Analytic Template further reduces the online computation to ~ 260 arithmetic operations by replacing ADMM with a first-order gain interpolation. Preliminary simulation results demonstrate that both variants maintain tracking accuracy comparable to online relinearization at substantially lower computational cost. Future work includes C implementation targeting the Crazyflie 2.1 STM32F405 microcontroller, FPGA synthesis of the Analytic Template datapath, and extension to wind-disturbance rejection.

REFERENCES

- [1] A. Alavilli, K. Nguyen, S. Schoedel, B. Plancher, and Z. Manchester, “TinyMPC: Model-predictive control on resource-constrained microcon-

trollers,” in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2024.

- [2] D. Mellinger and V. Kumar, “Minimum snap trajectory generation and control for quadrotors,” in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2011, pp. 2520–2525.
- [3] R. M. Murray, M. Rathinam, and W. Sluis, “Differential flatness of mechanical control systems: A catalog of prototype systems,” in *Proc. ASME Int. Mechanical Engineering Congress*, 1995.
- [4] M. W. Mueller, M. Hehn, and R. D’Andrea, “A computationally efficient motion primitive for quadcopter trajectory generation,” *IEEE Trans. Robotics*, vol. 31, no. 6, pp. 1294–1310, 2015.
- [5] M. Fliess, J. Lévine, P. Martin, and P. Rouchon, “Flatness and defect of non-linear systems: introductory theory and examples,” *Int. J. Control*, vol. 61, no. 6, pp. 1327–1361, 1995.
- [6] B. Stellato, G. Banjac, P. Goulart, A. Bemporad, and S. Boyd, “OSQP: An operator splitting solver for quadratic programs,” *Mathematical Programming Computation*, vol. 12, pp. 637–672, 2020.
- [7] S. Boyd, N. Parikh, E. Chu, B. Peleato, and J. Eckstein, “Distributed optimization and statistical learning via the alternating direction method of multipliers,” *Foundations and Trends in Machine Learning*, vol. 3, no. 1, pp. 1–122, 2011.
- [8] M. Faessler, A. Franchi, and D. Scaramuzza, “Differential flatness of quadrotor dynamics subject to rotor drag for accurate tracking of high-speed trajectories,” *IEEE Robotics and Automation Letters*, vol. 3, no. 2, pp. 620–626, 2018.
- [9] W. J. Rugh and J. S. Shamma, “Research on gain scheduling,” *Automatica*, vol. 36, no. 10, pp. 1401–1425, 2000.
- [10] J. S. Shamma and M. Athans, “Guaranteed properties of gain scheduled control for linear parameter-varying plants,” *Automatica*, vol. 27, no. 3, pp. 559–564, 1991.
- [11] G. A. Hewer, “An iterative technique for the computation of the steady state gains for the discrete optimal regulator,” *IEEE Trans. Automatic Control*, vol. 16, no. 4, pp. 382–384, 1971.
- [12] M. M. Konstantinov, P. Hr. Petkov, and N. D. Christov, “Perturbation analysis of the discrete Riccati equation,” *Kybernetika*, vol. 29, no. 1, pp. 18–29, 1993.
- [13] A. Bemporad, M. Morari, V. Dua, and E. N. Pistikopoulos, “The explicit linear quadratic regulator for constrained systems,” *Automatica*, vol. 38, no. 1, pp. 3–20, 2002.